Reproducibility Engineering
Summer Term 2026
Chair of Scalable Database Systems
Prof. Dr.-Ing. Stefanie Scherzinger, Stefan Klessinger, Christoph Köhnen

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

Exercise Sheet 7

Lab Sessions: June 18/19, 2026

In this sheet, we compare two database architectures using BenchBase[1], which is a popular benchmark harness for database management systems (DBMSs). Specifically, we compare an *embedded* and a *client-server DBMS*.

1 Preparation

For this exercise, we provide files in the RepEng FIMGit Repository (directory `LabSession7/`). Update your local repository that you cloned in the first lab session:

1. Open a terminal in your local repository.
2. Pull the latest changes using `git pull`.

2 Embedded DBMS

In this exercise, BenchBase and the embedded DMBS SQLite shall run in the same container. BenchBase executes the YCSB benchmark, which is a well-known benchmark.

2.1 Building an Image for BenchBase and SQLite

Outside the FIMGit repository, clone the BenchBase repository, change into the repository directory, and checkout commit `33c0047`:

Terminal Session bash

```
user@host$ git clone https://github.com/cmu-db/benchbase
user@host$ cd benchbase
user@host$ git checkout 33c0047
```

[1] https://github.com/cmu-db/benchbase

Create a `Dockerfile` **in the root directory of the cloned BenchBase repository**. The Dockerfile shall

- be based on `eclipse-temurin:23-jdk`,
- install Git, which is required by the BenchBase build process to obtain version information from the cloned repository,
- copy the BenchBase source code into the image,
- build the SQLite profile using the provided Maven Wrapper,
- extract `target/benchbase-sqlite.tgz` into `/benchbase`,
- use `/benchbase` as its working directory, and
- start BenchBase using `java -jar benchbase.jar`.

Afterwards, build the image from the resulting Dockerfile:

```
user@host$ docker build -t benchbase-sqlite .
```

2.2 Running and Evaluating the YCSB Benchmark

The configurations for the YCSB benchmark[2] are provided in the FIMGit under directory `LabSession7/config/sqlite/`. They define a read-only (`ycsb_read_only.xml`), a mixed read/write (`ycsb_mixed.xml`), and a write-only (`ycsb_write_only.xml`) workload.

Inside the FIMGit repository in directory `LabSession7/`, run the provided script `run_sqlite_benchmark.sh`, which creates a separate result directory for each workload and runs all benchmarks using the `benchbase-sqlite` image:

Terminal Session bash

```
# Linux/macOS
user@host$ ./run_sqlite_benchmark.sh

# Windows PowerShell
user@host$ .\run_sqlite_benchmark.ps1
```

Each execution creates a file ending in `.summary.json` in the corresponding result directory, containing the measured throughput under the key `Throughput (requests/second)`.

The FIMGit repository provides the Python script `plot_throughput.py`. This script extracts the throughput value from the `.summary.json` file in each workload directory and creates a bar chart containing one bar for each workload. The resulting plot is stored as `results/sqlite/throughput.pdf`.

[2] Derived from BenchBase's `config/sqlite/sample_ycsb_config.xml`.

The plotting script is executable with a parameter that states the DBMS for which the results should be plotted: `python plot_throughput.py <DBMS>`. We execute the script in a separate Docker container so that its Python and plotting dependencies are not included in the `benchbase-sqlite` image, whose size will be measured later.

Build the image from `Dockerfile.plot` provided in the FIMGit repository:

```
user@host$ docker build -f Dockerfile.plot -t benchbase-plot .
```

Run the plotting script inside a container. Mount the current directory `LabSession7/` into the container so that the script can read the benchmark results and store the generated PDF on the host:

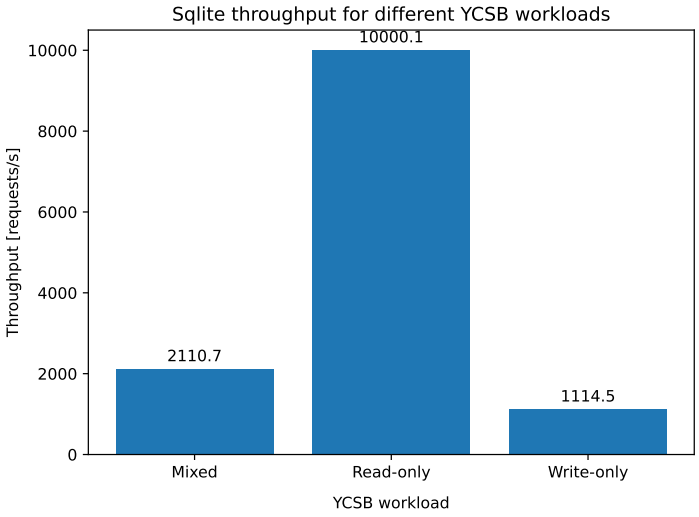
Terminal Session bash

```
# Linux/macOS
user@host$ docker run --rm -e MPLCONFIGDIR=/tmp/matplotlib \
    -v "$PWD:/work" -w /work benchbase-plot \
    plot_throughput.py sqlite

# Windows Git Bash
user@host$ docker run --rm -e MPLCONFIGDIR=//tmp/matplotlib \
    -v "$(pwd -W):/work" -w //work benchbase-plot \
    plot_throughput.py sqlite

# Windows PowerShell
user@host$ docker run --rm -e MPLCONFIGDIR=/tmp/matplotlib `
    -v "$($PWD.Path):/work" -w /work benchbase-plot `
    plot_throughput.py sqlite
```

The output should look similar to the following plot. The exact throughput values may differ depending on the system and its current load.



2.3 Size of the Binary Image

Measure the size of the binary image in bytes using `docker image inspect`.

2.4 Comparing the Image with a Prebuilt Runtime Image

The image `benchbase-sqlite` created in Exercise 2.1 contains both the tools required to build BenchBase and the files required to run it.

For comparison, pull a prebuilt runtime image containing BenchBase and SQLite:

```
user@host$ docker pull benchbase.azurecr.io/benchbase-sqlite
```

Measure the size of the prebuilt runtime image in bytes using `docker image inspect`.

Compare the result with the size of the image built in Exercise 2.1. Explain the differences.

3 Client–Server DBMS: PostgreSQL and BenchBase

We now create a client-server setup: PostgreSQL runs as server in one container; BenchBase runs as client in a second container. Docker Compose places both services in a common network, in which services can address each other by their service names.

3.1 Completing the Compose Setup

Complete the `docker-compose.yml` provided in the FIMGit repository under `LabSession7/`. It must satisfy the following requirements:

- PostgreSQL uses the image `postgres:18.4` and stores its data in a named volume.
- The database, user, and password are all called `benchbase`.
- PostgreSQL is reachable from the host at `127.0.0.1:54327`.
- BenchBase uses the image `benchbase.azurecr.io/benchbase-postgres`.
- The configuration and result directories are mounted into the BenchBase container to the directories `/benchbase/config/postgres/lab7` (read-only mount) resp. `/benchbase/results`.

3.2 Running and Comparing the YCSB Benchmarks

The following PostgreSQL configurations[3] are provided in the FIMGit repository in directory `LabSession7/config/postgres/`, for a read-only (`ycsb_read_only.xml`), a mixed read/write (`ycsb_mixed.xml`), and a write-only (`ycsb_write_only.xml`) workload.

They execute the same workloads and use the same benchmark parameters as the SQLite configurations in Exercise 2. The connection parameters differ because BenchBase accesses PostgreSQL over the Docker Compose network.

Inside the FIMGit repository in directory `LabSession7/`, run the provided script `run_postgres_benchmark.sh`, which, for each workload, creates a separate result directory, starts with a new PostgreSQL database, starts the PostgreSQL service, and executes BenchBase as a one-off Compose container:

Terminal Session bash

```
# Linux/macOS
user@host$ ./run_postgres_benchmark.sh

# Windows PowerShell
user@host$ .\run_postgres_benchmark.ps1
```

Each execution creates a file ending in `.summary.json` in the corresponding result directory, containing the measured throughput under the key `Throughput (requests/second)`.

[3] Derived from BenchBase's `config/postgres/sample_ycsb_config.xml`.

Analogously to Exercise 2, run the plotting script `plot_throughput.py` inside a container, here with parameter `postgres`:

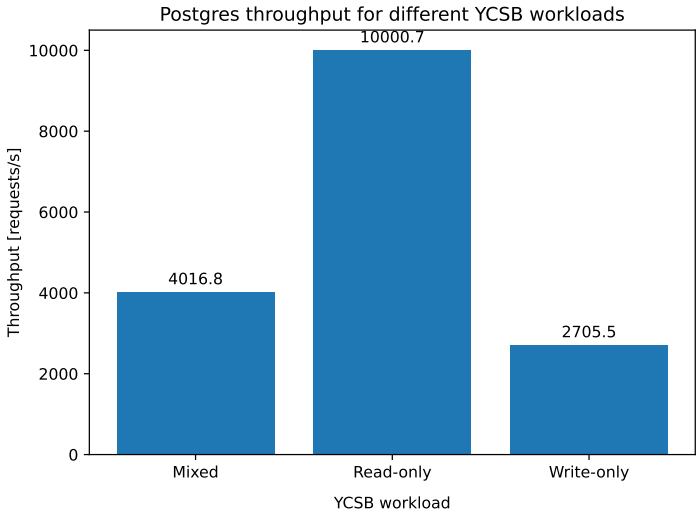
Terminal Session bash

```
# Linux/macOS
user@host$ docker run --rm -e MPLCONFIGDIR=/tmp/matplotlib \
    -v "$PWD:/work" -w /work benchbase-plot \
    plot_throughput.py postgres

# Windows Git Bash
user@host$ docker run --rm -e MPLCONFIGDIR=//tmp/matplotlib \
    -v "$(pwd -W):/work" -w //work benchbase-plot \
    plot_throughput.py postgres

# Windows PowerShell
user@host$ docker run --rm -e MPLCONFIGDIR=/tmp/matplotlib `
    -v "$($PWD.Path):/work" -w /work benchbase-plot `
    plot_throughput.py postgres
```

The output should look similar to the following plot. The exact throughput values may differ depending on the system and its current load.



Compare the resulting PostgreSQL plot with the SQLite plot created in Exercise 2. Base the comparison only on the measurements obtained on the same host system.

3.3 Size of the Client-Server Setup

Measure the size of the entire client-server setup, i.e., the sum of the BenchBase client image and the PostgreSQL server image in bytes using `docker image inspect`.

Compare the result with the size of the prebuilt runtime image measured in Exercise 2.4.

4 Database Architectures (Antwort-Wahl-Verfahren)

In this exercise, each question has exactly one correct answer. Mark the correct answer.

(a) Consider the following PostgreSQL setup:

PostgreSQL SQL

```
-- file_fdw exposes files from the PostgreSQL server's file system
-- as read-only foreign tables
CREATE EXTENSION file_fdw;
CREATE SERVER csv_files FOREIGN DATA WRAPPER file_fdw;

CREATE FOREIGN TABLE measurements (
    experiment_id INTEGER,
    runtime_ms    DOUBLE PRECISION
)
SERVER csv_files
OPTIONS (
    filename '/data/measurements.csv',
    format 'csv',
    header 'true'
);
```

Which statement is correct?

○ PostgreSQL imports the CSV data into its own storage when the foreign table is created.

○ Queries read the external file through the foreign-data wrapper.

○ Access to the tuples in the foreign tables can be accelerated by creating indexes on table `measurements`.

○ The CSV file can be safely deleted after executing the `CREATE FOREIGN TABLE` statement, this will not affect later queries.

○ None of these options

(b) MariaDB is a client-server DBMS. You want to benchmark MariaDB using the load testing tool `mariadb-slap`.

Which are suitable setups for measuring MariaDB performance?

Configuration A:

Dockerfile

```
FROM mariadb:11.8
COPY run_benchmark.sh /run_benchmark.sh
# run_benchmark.sh also runs mariadb-slap
CMD ["/run_benchmark.sh"]
```

Configuration B:

docker-compose.yml

```
services:
  mariadb:
    image: mariadb:11.8
    environment:
      MARIADB_ROOT_PASSWORD: benchbase

  benchmark:
    image: mariadb:11.8
    depends_on:
      - mariadb
    entrypoint: ["mariadb-slap"]
    command:
      - "--host=mariadb"
      - "--user=root"
      - "--password=benchbase"
      - "--auto-generate-sql"
```

○ Only configuration A ○ Only configuration B

○ Both configurations ○ Neither configuration

(c) Assume that a DBMS manages a table

```
CREATE TABLE items (
    id    integer PRIMARY KEY,
    value integer NOT NULL
);
```

Consider the following queries:

I. `SELECT id, value FROM items ORDER BY id;`

II. `SELECT id, value FROM items;`

III. `SELECT id, value FROM items LIMIT 1;`

IV. `SELECT id, value FROM items ORDER BY id LIMIT 1;`

V. `SELECT id, value FROM items ORDER BY value LIMIT 1;`

How many of these queries are guaranteed to produce reproducible results?

○ 0 ○ 1 ○ 2 ○ 3 ○ 4 ○ 5